

Reproducibilidad y ejecución del pipeline

El proyecto fue implementado como un pipeline reproducible en Python que integra todas las etapas necesarias para entrenar y evaluar un agente de Reinforcement Learning aplicado al trading de BTCUSDT.

El flujo completo incluye:

1. Carga y preprocesamiento de datos
2. Ingeniería de variables (features)
3. Construcción del entorno de trading
4. Entrenamiento del agente PPO
5. Evaluación walk-forward
6. Cálculo de métricas
7. Generación automática de artifacts

Todo el pipeline puede ejecutarse desde un único archivo principal.

Punto de entrada (Entrypoint)

El archivo principal del proyecto es:

`main.py`

La ejecución completa del proyecto se realiza mediante:

`python main.py`

Al ejecutarse, este archivo:

- carga los datos históricos,
- genera features,
- construye el entorno,
- entrena un modelo PPO para cada fold,
- evalúa el desempeño,
- y genera todos los artifacts requeridos.

Reproducibilidad y semillas aleatorias

Para garantizar reproducibilidad, se fijaron semillas aleatorias en todas las librerías relevantes:

SEED = 42

```
np.random.seed(SEED)
random.seed(SEED)
torch.manual_seed(SEED)
```

Además, el agente PPO se inicializa utilizando la misma semilla:

```
model = PPO(..., seed=SEED)
```

Esto permite reducir variaciones entre ejecuciones y mantener resultados consistentes bajo las mismas condiciones de hardware y software.

Estructura del pipeline

El código está organizado en distintas etapas funcionales.

1. Carga de datos

Función principal:

```
load_data()
```

Responsabilidades:

- cargar el dataset BTCUSDT OHLCV,
- limpiar filas inválidas,
- convertir timestamps,
- ordenar cronológicamente los datos,
- convertir columnas numéricas.

2. Resampleo temporal

Función:

```
resample_4h()
```

Responsabilidades:

- transformar velas de 1 hora a 4 horas,

- reducir ruido de mercado,
- disminuir sobre-operación del agente,
- estabilizar parcialmente el entrenamiento.

3. Ingeniería de variables

Función:

`create_features()`

Variables generadas:

- retornos logarítmicos,
- medias móviles,
- desviaciones respecto a medias móviles,
- volatilidad histórica,
- rangos high-low,
- z-scores de volumen,
- z-scores de actividad de trading.

Todas las variables se calculan únicamente con información pasada, evitando lookahead bias.

4. Entorno de trading

Clase principal:

`BTCTradingEnv`

Responsabilidades:

- definir el espacio de estados,
- definir el espacio de acciones,
- simular operaciones,
- aplicar costos de transacción,
- calcular recompensas,
- registrar operaciones realizadas.

El entorno sigue una formulación semi-Markov donde las recompensas se materializan únicamente al cierre de la operación.

5. Entrenamiento walk-forward

El pipeline entrena un modelo PPO independiente por cada fold temporal.

Esquema utilizado:

Fold Entrenamiento Test

1	2017–2021	2023
2	2017–2022	2024
3	2017–2023	2025
4	2017–2024	2026

Cada modelo es completamente out-of-sample respecto a su periodo de prueba.

6. Evaluación

Para cada fold se calculan:

- retorno total del agente,
- benchmark buy-and-hold BTC,
- benchmark SP500,
- Sharpe ratio,
- maximum drawdown,
- número de trades.

Posteriormente se agregan medias y desviaciones estándar.

Artifacts generados automáticamente

La ejecución completa del pipeline produce los siguientes archivos.

Exploratory Data Analysis

Archivo:

btcusdt_eda.pdf

Contiene:

- identificación de regímenes,
- distribuciones de retornos,
- análisis de autocorrelación,
- pruebas de estacionariedad.

Análisis de features

Archivos:

feature_analysis_fold_1.csv

feature_analysis_fold_2.csv

feature_analysis_fold_3.csv

feature_analysis_fold_4.csv

Contienen relaciones feature-target calculadas únicamente sobre datos de entrenamiento.

Curvas de entrenamiento

Archivos:

training_curves_fold_1.png

training_curves_fold_2.png

training_curves_fold_3.png

training_curves_fold_4.png

Muestran la evolución de recompensas durante el entrenamiento PPO.

Resultados de validación

Archivos:

validation_results_fold_1.csv

validation_results_fold_2.csv

validation_results_fold_3.csv

validation_results_fold_4.csv

Incluyen resultados de búsqueda y evaluación de hiperparámetros.

Curvas de equity

Archivos:

equity_curves_test_fold_1.png
equity_curves_test_fold_2.png
equity_curves_test_fold_3.png
equity_curves_test_fold_4.png

Comparan:

- estrategia RL,
- buy-and-hold BTC,
- benchmark SP500.

Logs de operaciones

Archivos:

trade_log_fold_1.csv
trade_log_fold_2.csv
trade_log_fold_3.csv
trade_log_fold_4.csv

Incluyen:

- fecha de entrada,
- fecha de salida,
- precios,
- PnL bruto,
- costos,
- PnL neto,
- motivo de cierre.

Resumen de métricas

Archivo:

metrics_summary.csv

Incluye:

- métricas por fold,
- promedios agregados,
- desviaciones estándar.

Librerías utilizadas

Principales dependencias del proyecto:

- Python 3
- Stable Baselines3
- Gymnasium
- PyTorch
- NumPy
- Pandas
- Matplotlib
- yfinance

Notas sobre reproducibilidad

Aunque las semillas aleatorias fueron fijadas, pequeñas variaciones pueden surgir debido a:

- diferencias de hardware,
- versiones de CUDA/cuDNN,
- implementaciones BLAS,
- sistema operativo.

Sin embargo, el comportamiento general y las conclusiones del proyecto permanecen consistentes entre ejecuciones.